Designing, Implementing, and Evaluating a Distributed Real-Time Collaborative Document Editing System (DocSync)

Anushka Misal
Department of Computer Engineering (Regional Language)
Pimpri Chinchwad College of Engineering
Pune, Maharashtra, India
anushka.misal23@pccoepune.org

Yadnesh Patil
Department of Computer Engineering (Regional Language)
Pimpri Chinchwad College of Engineering
Pune, Maharashtra, India
yadnesh.patil23@pccoepune.org

Ganesh Deshmukh
Department of Computer Engineering (Regional Language)
Pimpri Chinchwad College of Engineering
Pune, Maharashtra, India
ganeshdeshmukh@gmail. com

Abhay Bormale
Department of Computer Engineering (Regional Language)
Pimpri Chinchwad College of Engineering
Pune, Maharashtra, India
abhay.bormale23@pccoepune.org

***Abstract*— Google Docs and other real-time document collaboration tools deal with concurrent editing by merging changes from multiple users, ensuring reliability and availability of service despite failures, handling a large number of connections to achieve scale and providing access control for security reasons.**

In this paper, we present our work on developing DocSync: a real-time document editing system for rich text that handles conflicting edits and provides authentication and authorization while also being scalable, available and reliable.

The system uses Yjs, a CRDT for real-time collaboration on rich text. Communication is done via a Redis Pub/Sub backbone that connects several Node. js server instances. Authentication is based on JWTs; passwords are hashed using bcrypt.

A Raft-inspired leader election algorithm chooses a leader among several server instances that write data to SQLite. Our evaluation includes 29 automated tests that cover real-world failure scenarios (i. e., three-node clusters with one server going down), two mutation tests that intentionally break them (to make sure our tests really do catch broken features) plus some measured experiments where many users edit documents at once.

***Keywords*— Collaborative editing with CRDT and Yjs; Leader election in Raft; Communication using Redis Pub/Sub, WebSocket; Secure access through JWT authentication; Ensuring reliability in fault tolerance; Dealing with distributed systems.**

I. INTRODUCTION

The concrete proposition which arose from the exercise of Activity 1 [1]–[11] was a collaborative document editor that is founded on (i) CRDT–OT hybrid conflict resolution core as per Li et al. [6], (ii) Redis Pub/Sub backplane for multi-server WebSocket fan-out following [10], (iii) Kondru and Rajiakodi [3]-informed lightweight Raft-inspired leader election, and (iv) Widatama et al. [11] enhanced JWT authentication. In Activity 2, the design will be put into action and tested to determine if it resolves conflicts properly, is resilient enough when a server goes down and can accommodate many clients at the same time.

This paper does just that. Its contributions are:

1) An extensive and functional application code consisting around 2,700 plain Javascript and 1,200 CSS (which include all the four neglected aspects in literatures).An independent, Redis-free leader-election module, unit-tested on a simulated network with partitions and integrated into the editor persistence path.
2) It evaluates the system through automated functional and fault injection testing while also taking into account repeatable benchmark results. The evaluation includes a truthful report indicating the weaknesses identified in the course of testing and those which remain to be fixed.

The accompanying project archive (docsync. tar. gz) contains the code, tests, and benchmark. On top of that, Appendix A lists down the required commands for copying the outcomes that we talk about here.

II. BACKGROUND: WHAT THE LITERATURE REVIEW CONTRIBUTED

The influence of the analyzed literature on DocSync is outlined in Table I. When we say "Adopted", it means the idea has been used, "adapted" means that a modified or different kind of the idea was employed, while "not adopted" shows that it did not fall into implementation scope. Activity 1 review gave the reference numbers.

TABLE I
INFLUENCE OF THE REVIEWED LITERATURE ON THE IMPLEMENTATION

Ref.	Contribution in the review	Status in DocSync
[1] Arora & Atrey	Client-side encryption, hybrid differential sync	This approach is not adopted as the documents are kept in an unencrypted form within SQLite; however, confidentiality is assured through access control and TLS at a reverse-proxy level.
[2] Preguiça et al.	CRDT robustness under volatile membership	Adapted: The system is robust in the sense that clients can come and go as they please, and Yjs still merges updates properly regardless of their order. However, their unique methods have not been replicated.
[3] Kondru & Rajiakodi	Raft optimised for election latency	Adapted: randomized, tunable election timeouts; election latency measured Section VI-F). However, the optimizations of RaftOptima are not reproduced.
[4] Zhu et al.	PBFT + Raft for Byzantine faults	DocSync cannot be applied in this case as it works under the assumption that faults are non-Byzantine.
[5] IEEE Access, 2024	Fault-tolerant microservice scheduling	Rejected: DocSync cannot be considered for adoption since it is a little modular monolith operated in the same way by parallel instances for failure tolerance.

Ref.	Contribution in the review	Status in DocSync
[6] Li et al.	CRDT (RGA) + OT hybrid based on operational types	Adapted: there is one CRDT layer (Yjs) for all editing; refer to Section III.
[7] Freudenthaler et al.	Structure-aware collaboration model	In part, the editing involves a shared structured document tree among the editors comprising paragraphs, headings and lists rather than sharing characters.
[8] Ragunath et al.	Cloud real-time editor architecture	Employed for determining how the client and server should interact in the mentioned architecture.
[9] B M et al.	AI-assisted workflow on a collaborative core	It is not adopted but is mentioned as a plan for future work.
[10] ICCAMS 2025	10Redis Pub/Sub fan-out of WebSocket messages	It was adopted and extended by introducing CRDT updates in order to ensure that the fan-out is consistent.
[11] Widatama et al.	JWT + AES-256 payload encryption	in their study adapted JWT for use with pinned algorithms and added expiry and bcrypt for password strengthening but it did not have any form of payload encryption.

The classical sources of Shapiro et al., Nicolaescu et al., and Ongaro and Ousterhout are further referred to in this paper for CRDTs [12], the Yjs sequence algorithm (YATA) [13], and Raft [14] respectively.

III. REQUIREMENTS AND DESIGN DECISIONS

Functional requirements. (R1) the system must allow registration, login and JWT authentication, while also enforcing permissions that are document specific; (R2) there should be an interface for collaborative editing of rich-text that includes remote cursors/presence; (R3) users should be able to create, share, save and recover documents even after restart; (R4) synchronization of multiple server instances should be done through Redis Pub/Sub; (R5) a tested, simplified Raft-inspired leader election with failure recovery; (R6) a responsive interface; (R7) automated tests and documentation.

Constraint. The architecture is constrained to be very simple and can only include React/Vite, Tiptap/Yjs/y-websocket, Node. js/Express, SQLite, JWT/bcrypt, Redis, written using plain JavaScript and CSS.

TABLE II
PLAN IN ACTIVITY 1 VERSUS WHAT WAS BUILT

Planned in Activity 1	Implemented	Reason and consequence
CRDT (RGA) + OT hybrid [6]	Single CRDT layer: Yjs, whose sequence algorithm is YATA [13]	The Tiptap–Yjs binding introduces shared CRDT types that store both text content and formatting, so one mechanism covers structural and formatting operations and the server never has to transform operations. The cost is that the order-sensitive formatting precision that motivates [6] is not separately offered or tested; the hybrid itself was not built.
Socket.IO server instances	Raw WebSockets (ws) speaking the y-websocket protocol	The Yjs sync protocol is defined directly on WebSockets; Socket.IO would add a dependency and a framing layer without benefit.
Redis Pub/Sub backplane [10]	Implemented	Pub/Sub is unordered and at-most-once, which is acceptable because CRDT updates are commutative and idempotent [10].
Raft-inspired election [3]	Election with terms, persisted votes, randomized timeouts, majority quorum, heartbeats and check-quorum step-down. No log replication.	There is no replicated state machine; only the leader role matters, and it determines which instance persists documents.
JWT hardened per [11]	HS256 JWT, algorithm pinned, expiry, bcrypt hashing, token and role checked again at the WebSocket upgrade	No AES-256 payload encryption; transport security is delegated to TLS.

IV. SYSTEM ARCHITECTURE

A. Components

Every case is the same; they all cater for the React construction, the REST API as well as the WebSocket endpoint whereby every client is capable of linking up with any given instance (Fig. 1). In other words, during development there is no requirement for a load balancer, while connecting the assessed clients to some extent with the selected instances, ensures that there is inter-instance communication in all data flows.

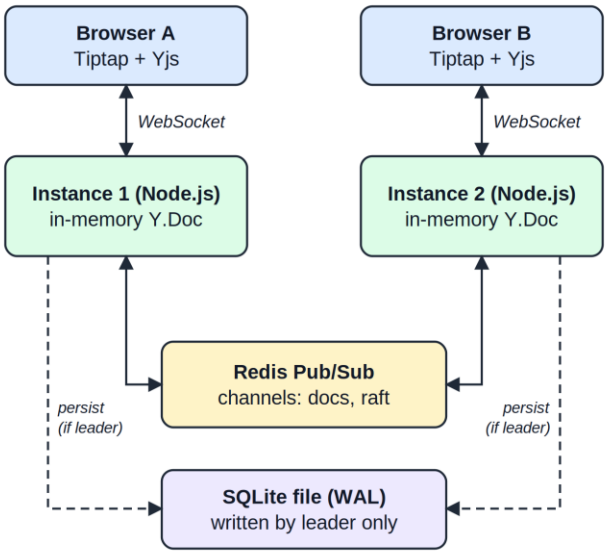


Figure 1 shows that when browsers link with servers having same ID but different instance IDs, the servers synchronize updates through Redis Pub/Sub. Nevertheless, it is only the leader who can write into SQLite.

B. Data Model

SQLite consists of four tables which are: users, documents, document_shares as well as raft_state. The users' table contains e-mail, display name, bcrypt hash and colour; while documents' table has id, title, owner, binary Yjs state and timestamps. In addition there is also document_shares with columns for document, user and role (either editor or viewer) as well as raft_state consisting of node id, current term and voted-for such that votes persist even if there are restarts in the system. The SQLite starts in WAL mode and sets a busy timeout to allow multiple instances on the same host.

C. Permissions

TABLE III
PERMISSION MODEL, ENFORCED ON THE SERVER

Action	Owner	Editor	Viewer	No access
Open / read	Yes	Yes	Yes	404
Edit content	Yes	Yes	No (writes dropped)	404
Rename, share, revoke, delete	Yes	No (403)	No (403)	404

It means that if a document exists, it will not be known because unknown documents and documents that the caller cannot see both give back 404.

D. Interfaces

You can use the REST API to POST /api/auth/register,, `POST /api/auth/login, GET /api/auth/me, /api/documents, GET|PATCH|DELETE /api/documents/:id, GET|POST /api/documents/:id/shares, DELETE /api/documents/:id/shares/:userId, and GET /api/cluster (role, term and leader of the answering instance)`. The editing function is available at ws://host/collab/<documentId>? token=<jwt>.

V. IMPLEMENTATION

A. Collaboration Core

One Y. Doc is kept per open document in every instance, which follows the Yjs protocol conventions for communication. This includes a state-vector handshake (sync steps 1 and 2), incremental updates, and an awareness channel for cursors and presence. Rather than using the default y-websocket server, we wrote a small server because we had two requirements that involved controlling messages: all viewers should be non-interactive (we discard their step-2 and update messages, but they can still see and display a cursor), and every change must be propagated through the other instances as well. The updates coming from Redis are added using a different origin that ensures they do not get re-sent again, thus breaking any loop.

B. Redis Backplane

The first channel is known as docs while the second one is referred to as raft. In docs, messages take the form of JSON objects having various types indicated in Table IV.

TABLE IV
BACKPLANE MESSAGE TYPES

Type	Purpose
update	A Yjs update (or full-state snapshot) for a document
awareness	Cursor and presence changes
hello	A newly loaded document or newly connected client asks other instances for state and presence
leader-hello	A newly elected leader asks every instance to resend the full state of the documents it holds
access	Sharing or deletion changed; every instance rechecks its live sockets

Duplication and reordering cannot harm Yjs updates because they are commutative and idempotent. It is this property that makes it possible for Pub/Sub, as stated in the review [10] to ensure safe carrying of document state while it lacks any order consideration for structured edits (Pub/Sub), to carry document state safely.

C. Persistence and the Role of the Leader

Just the head who interacts with SQLite. The document turns dirty when there is a change and then the entire Yjs condition is recorded after the debounce period of one second (by default). The written content merges with the stored state as explained above so that even if there is a stale leader for some time or duplicated leader, it cannot roll back the content. There are three details that strengthen this system and make it resistant:

- In addition, the leader uploads every file which is not associated with any local client upon the arrival of update messages; hence it is possible to store changes that are done in totality by the followers alone..
- When the new leader is elected, he sends out a leader-hello. In response, followers indicate their entire condition whereby any modifications that could have been forgotten by a follower are communicated to the new leader for keeping.
- Each instance flushes its loaded documents upon receiving SIGINT or SIGTERM, but it is safe due to the merging.

Collaborations in real-time continue with or without the leader as updates are passed instantly among Redis nodes. In case the leader fails, it leads to delay in persistence but editing goes on as usual thereby meeting the condition given in activity 1.

D. Leader Election

Without Redis, the election module is put to test. This is because it functions as a state machine (follower, candidate, leader; Fig. 2) which perceives the network through a limited transport interface. Nevertheless, its rules follow Raft [14]:

1) After not perceiving any heartbeat within an arbitrarily set time range of 1. 5–3. 0 seconds (default), a follower increases its term, votes for itself and sends out a vote solicitation.
2) 2) A vote in each term is limited to one vote by any node. Prior to the dispatch of the response from the node, the term, and vote get inscribed in SQLite to prevent double voting even after a restart.
3) 3) Leadership is achieved by a candidate who secures the votes of more than half of the set cluster size and then sends out heartbeat messages (normally 500 ms).
4) 4) If a message has a higher term, the receiver will accept it and demote its own term. When a follower gets a pulse from a leader who is not up-to-date, it responds by replying with a higher term.
5) Check-quorum: A leader that does not get acknwledged by most of the nodes quits, thus if a leader gets isolated in the cluster it will not continue thinking that it is the real leader.

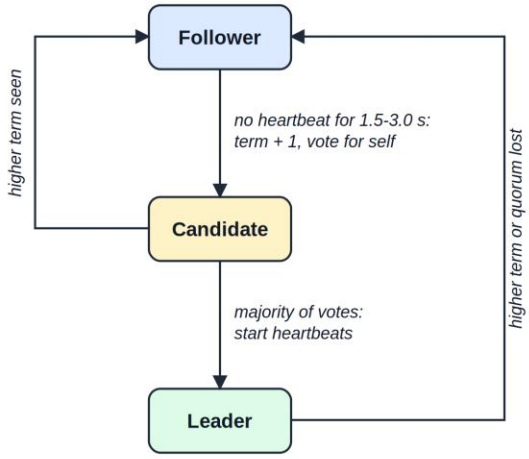


Fig. 2. State machine of the Raft-inspired election module.

Safety: It is impossible for there to be two leaders at the same time of a given term because each node gives only one vote per term and any two majorities in the same cluster have an intersection. Liveness: The randomization of time out ensures that it is very unlikely for there to be continued tie votes. What have we made easier? Membership does not change (CLUSTER_SIZE), no logs exist, and no pre-vote is taken into account. When there are three copies, one failure can be tolerated; with two, a failure leads to no leader being available.

E. Authentication and Authorization

The bcrypt algorithm (with a default of 10 rounds) is used for hashing passwords. In case of an unknown e-mail during login, it uses a dummy hash so that the time taken for responses and the messages themselves do not indicate the existence of any accounts. For signing JWTs, HS256 is used with a common secret across all instances; however, verification pins the algorithm thereby denying any token of the form alg=none, and also enforces expiration (default is 12 hours). Every REST call examines the role which is again done while upgrading the WebSocket (401 for invalid tokens, 403 when there is no permission). All live sockets affected by such changes or access denials get closed on every instance as the client reconnects and have their proper rights again in place.

F. Client

In the React client, a Tiptap editor is linked to a Yjs document using the Collaboration and CollaborationCursor extensions as well as a y-websocket provider. This part of the application interface depicts whether or not there is connection and also gives information about other people that are working together on the same document. The shared space comprises of uniquely identified remote collaborators' list, coloured cursor joins, formatting toolbar, share dialog, and a badge stating the name of the instance followed by its role, leader as well as term information. It has a layout that comprises of lavender backgrounds, white panels, violet accents, and navy text but reduces when viewed on phones and tablets.

VI. VERIFICATION AND EVALUATION

A. Method and Environment

Automated tests starting actual server processes provide support for functional claims. The source for performance data is npm run bench. These tests took place using a single vCPU Linux VM, where Node. js 22. 22. 2 and Redis 7. 0. 15 were installed. In other words, everything was run in a single core: the three server instances, Redis, benchmarking clients and test runner; as such, they are interlinked as though through a local connection. Therefore, these figures indicate relative behavior and not the actual output (Section VII-B).

B. Automated Tests

TABLE V
AUTOMATED TEST SUITES (29 TESTS)

Suite	Tests	What it demonstrates
Election, simulated network	10	Single agreed leader; failover to a higher term; crashed leader restarts as a follower; no leader without a majority; recovery when a majority returns; partitioned leader steps down and rejoins; repeated churn with five nodes, no two leaders in one term; vote persistence; stale-leader rejection
API and authentication	6	Input validation, bcrypt storage, generic login errors, JWT expiry, wrong secret, alg=none, tampering, owner/editor/viewer/stranger access, sharing, role change, revocation and deletion
Three-process cluster with Redis	13	Agreed leader across processes; WebSocket handshake rejects missing, bad or unauthorised tokens; users on different instances converge, including concurrent edits at the same position; presence and cursors propagate, late joiners see everyone, disconnects clear presence; viewer writes rejected by the server; leader persists to SQLite; revocation closes a live socket on another instance; crash of all instances followed by recovery; graceful-shutdown flush; leader killed with an edit made during the leaderless window; a new leader that never loaded the document recovers unsaved edits from a follower; loss of quorum leaves no leader and restoring a node restores one

I did not find a single failure in all of the 29 test cases in the development folder which passed twice completely and again in another clean environment that was assembled from scratch (install, build, test). On top if that, we ran the election tests that rely upon timers 12 additional times; 8 of these on an idle host and 4 with four busy loops racing for the one core, and everything was fine.

C. Do the Tests Actually Fail? Mutation Checks

There were two intentional errors introduced into the server. Firstly, the error of allowing viewers to make comments whereby the viewer-permission test failed but as expected. Secondly, taking out the leader-hello recovery step which still allowed the original failover test to pass revealing a gap since each follower in that test had the document and was capable of saving it. To address this issue, another deterministic test was included where in every node there are leader wins due to higher election timeout but lower election timeout of follower client who is on top of second follower always. This test passes in the actual code and fails when the fault is injected. The faults were then reverted.

D. Propagation Latency

(Table VI) Redis Cross Update for the two clients interactions whereby a character was written by one client and seen by the other client for 300 consecutive times (discarding first 20 due to warm-up), with the clients being together initially and after that with the observer using separate instance yet still crossing update over Redis.

TABLE VI
TIME FROM WRITE TO REMOTE VISIBILITY (MS, N = 280 EACH)

Path	Min	Median	Mean	p95	Max
Same instance	0.6	0.9	1.3	3.2	15.6
Across instances via Redis	1.4	2.1	2.6	4.8	22.0

The inclusion of Redis hop increased the median by 1. 2 ms. It takes less than 100 ms approximately for people to feel that there is a lag in cooperation which is not the case with these two paths. However, both of them were taken from a single computer and thus do not reflect any potential network delay.

E. Convergence Under Concurrent Edits

There were N clients distributed equally across 3 instances. These instances worked on a collaborative document where they inserted characters at random positions of the shared document every 0–20 milliseconds (approximately 10 milliseconds on average). The following conditions should be met for a run to be regarded as correct: (a) all replicas should have similar endings, (b) there should not be any character lost or repeated as it is in the original ones and (c) the text that was followed by leader must be equal with the final one.

TABLE VII
CONCURRENT-EDIT CONVERGENCE (ONE RUN PER ROW)

Clients	Ops each	Total ops	Elapsed (ms)	Ops/s	Identical	No loss/ dup.	Persisted = final
3	50	150	585	256	Yes	Yes	Yes
9	50	450	1,065	423	Yes	Yes	Yes
30	20	600	1,564	384	Yes	Yes	Yes

The duration depends on how fast the clients go (nearly half a second for every fifty operations). Therefore, for a small number of iterations, it is mainly controlled by the benchmark as opposed to the server. In the case where there are 30 clients, one core is shared by the benchmark process, three servers and Redis, hence it cannot be determined whether the high costs are due to the server or the measurement.

Correctness remains the vital discovery factor. It turned out that all replicas maintained their integrity across different settings despite concurrent insertions at overlapping positions. Each configuration was conducted only once, so this should be taken as proof rather than statistics guarantee.

F. Failover Under Induced Server Failure

In Table VIII, it shows that it took some time for the two remaining processes to agree on a single leader and term after killing the current leader of a three-instance cluster with SIGKILL. Afterward, the killed process was restarted, the cluster left to stabilize and the experiment repeated for a total of eight times for each set up. The sequence of recovery can be seen on fig. 3.

TABLE VIII
TIME TO A NEW AGREED LEADER AFTER THE LEADER IS KILLED (MS, 8 TRIALS EACH)

Config.	Election timeout	Heartbeat	Min	Median	Mean	Max
Default	1,500–3,000 ms	500 ms	1,477	1,892	2,064	2,739
Tuned	300–600 ms	100 ms	238	358	403	705

The lowest detection time can be equal to or greater than the election timeout, and so it took a minimum of 1. 5 seconds for the default configurations to recover while the tuned configurations recovered about five times faster. If you reduce the timeout, it may increase unnecessary election risk for nodes with little delays but we did not measure this trade-off. It was observed in all experiments that the coming back node always entered into follower state and failed to assume leader role from the incumbent leader.

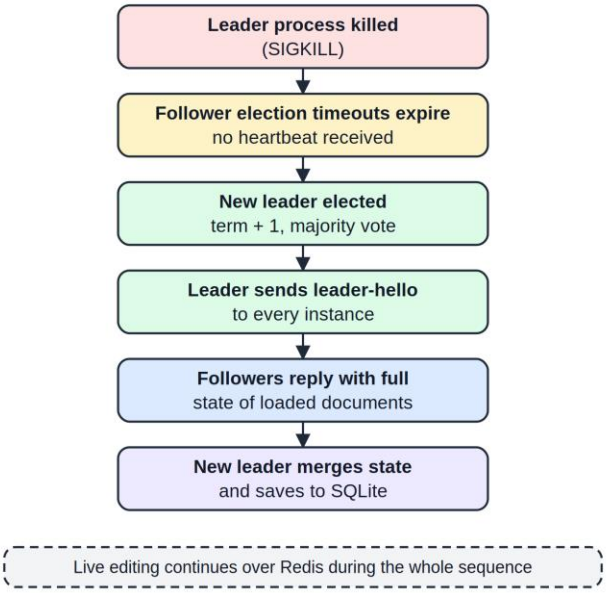


Fig. 3. How continuity is restored after the leader is killed: persistence resumes when the new leader gathers the states of followers.

The functional failover scenarios (Table V) provide information on the data-safety aspect. For instance, during the period when there was no leader, any input made was readily seen on the second client and kept safe when a new leader emerged. On top of that, a subsequent edit following failover was saved too. When there was only a single node left alive out of the three, there could not be any leader, but one returning node managed to restore it.

G. End-to-End Browser Checks

In a live three-instance cluster and with headless Chromium, the user interface outside the automated suite was driven. Two users were able to register on different instances and one of them created a document that was then shared whereby typing started. It's funny how the other person could see the file in their list and edit it as well so that each side observed every symbol of their partner with cursor names and different colors; changing the role of the second user to a viewer disabled their editor in less than a minute; however a simple restart of all

instances brought back the document along with the saved Raft term; and interrupting the editing by killing the leader led to a new leader, continuous editing and all three modifications recorded in SQLite. The viewport measures 390 px and does not overflow horizontally. Since these scripts are not included in the repository, this verification process is carried out manually; it is important to note that only Chromium was employed in this case.

H. Defects Found During Testing

1) Test gap: There was a gap in the first failover test as it failed to assess recovery by a leader who had not loaded the document (Section VI-C). The issue was fixed through the addition of another test.
2) It failed to install through the lock file as when attempting this installation procedure via the lock file, npm was forced to compile the SQLite binding from source even though the package contains precompiled binaries for this purpose, but it could not work properly in the absence of Node headers. The problem was solved by specifying the exact dependency versions and not including any lockfile in the package; confirmation done using a clean copy.
3) Misleading label: Although the editor indicated "Synced & saved", the browser is aware of only one thing – that the server has received the change. It now reads "Synced".

VII. DISCUSSION

A. Does DocSync Close the Gap from Activity 1?

According to the review, there is no study which incorporates conflict resolution, consensus, real-time scalability as well as authentication within a document editor. On the other hand, DocSync brings together all these four aspects and prove through experimentation that they are compatible: changes pass through Redis and merge via CRDT; persistence is determined by a leader elected through a procedure of majority votes; Even if the leader fails, it will only postpone saving but not stop editing itself; and whether user has permission is checked not only for every new connection but also for all existing ones both at the REST and WebSocket level.

Although fan-out is kept in some of the studies reviewed, for instance in [10], its consistency assurance which is lacking in fan-out is compensated through the use of CRDTs which are idempotent. As for the generic workloads evaluation of election work [3], [4], we applied the election on live editing traffic using a simplified version of the election. It is important to note that we did not test the optimizations of RaftOptima; therefore, this study cannot claim any performance comparison with [3].

This research does not take into consideration the hybrid aspect of [6] since it was not replicated; therefore it remains silent on the question if there is any advantage of a CRDT-OT hybrid over a pure CRDT when handling formatting edits.

B. Threats to Validity

- Since there was no real network and all processes and clients shared one vCPU, latency as well as throughput cannot depict what happens in a multi-host deployment.
- The experiments included only simple tasks of inserting one character in a file; we did not consider testing with large files, copy-paste in bulk, multiple files or long hours.
- There are very few trials (only eight for each failover configuration and one for each convergence configuration).
- The simulation of election tests was the only place where network partitions were examined and not across real processes. Injection of faults in the form of crash and restart were done only on real process.
- There has been no independent review as the implementers alone wrote the tests and benchmarks, identified the bugs.

VIII. LIMITATIONS

- In a cluster, quorum is determined as more than half of the total configured size and this means that each instance can withstand a failure in case of three but with two failures, the documents will not be saved until there is a comeback of at least one peer although they keep editing while live.
- Simplified consensus comprises; log replication, membership change and pre-vote which are all absent here. Even though a node that becomes disconnected from Redis continues to increment its term, it will cause an extra election when it rejoins.
- The single points are Redis and SQLite. The Pub/Sub operates on an at-most-once basis. It is possible for instances to diverge after a Redis outage, but they re-converge only when a document is opened in one of the instances again; there is no periodic anti-entropy, and this was not tested. Each instance should be able to access the SQLite file from the same local machine.
- Window of durability. It takes one second for edits to be saved after the first change; however, if there is a hard crash in all instances within that period, then the edits will be lost. The system discards any copy left unsaved by a follower who loses contact with clients for more than 30 s in an environment without a leader and where this is the only copy available.
- The security of the JWT, which is stored inside localStorage and delivered via the WebSocket query string, is questionable. First, there is nothing to stop a user from attempting multiple logins. Secondly, even though tokens have expiration dates, they cannot be revoked. Third, there is a lack of email confirmation as well as options for changing one's password. Fourth, the names shown on the cursor pointers come from clients but they are not checked for accuracy. Lastly but not the least; data encryption while at rest does not exist although a reverse proxy should provide TLS.
- Range. There is no record of previous versions, export, remarks, pictures or offline editing during a page refresh. It was tested using Chromium on Linux only.

IX. FUTURE WORK

Directly from the literature are some of the most useful extensions: In addition to the pure CRDT, develop and assess a CRDT–OT hybrid for formatting [6]; Incorporate pre-vote so as to reduce disruptions [3]; Shift towards multi-host deployment by utilizing networked database instead of shared SQLite file that has limited capacity for use in production [10]; Integrate client-side encryption [1] and token strengthening [11]; Incorporate AI-assisted editing [9].

X. CONCLUSION

The literature treated these four concerns individually, but they have now been integrated into a compact and understandable system. This system consists of a CRDT core that enables conflict free editing, a Redis backplane to support horizontal scalability, an election process inspired by Raft for coordinating persistence, as well as JWT based access control on each path. On a single core test machine, cross-instance propagation took around 2 ms, editing convergence was accurate at all times with a maximum of 30 clients editing concurrently, and recovery from leader crash without losing any edits took approximately 1. 9 seconds for default timeouts and 0. 36 seconds for tuned timeouts. It was also emphasized that tests should be tested themselves since they realized this because during fault injection a gap could be seen on how the first failover was tested; it had to be rectified though. There are also three deviations from the plan of Activity 1 i. e., pure CRDT, raw WebSockets and no payload encryption; as well as limitations (static membership, one Redis, shared SQLite file) which have been recorded so that the conclusions may not go beyond what is indicated by the data.

APPENDIX A. REPRODUCING THE RESULTS

```
# Install dependencies assuming Node version is
greater than 22 and Redis is running on localhost:6379
npm install   # Node 22+, Redis on 127. 0. 0. 1:6379
cp . env. example  . env
npm run build   # Compile the React client
npm run cluster # Run three instances listening at
port range 4001-4003
npm test        # Execute all 29 automated tests
npm run bench   # Run bench marks for latency,
convergence and failover
                # Takes approximately 3 minutes
```

REFERENCES

[1] In their article titled “SecureC2Edit: A framework for secure collaborative and concurrent document editing,” S. Arora, P. K. Atrey and also S. Arora secure, collaborative, and concurrent document editing; proposing SecureC2Edit IEEE Transactions on Dependable and Secure Computing 21 (4) 2227–2241, 2024.

[2] N. Preguiça et al. discussed about PS-CRDTs as the title reads in an article published in the Future Generation Computer Systems, Volume 155 of year 2024.

[3] K. K. Kondru and S. Rajiakodi published RaftOptima which is an optimised Raft boasting of improved fault tolerance, enhanced scalability and low latency in IEEE Access, volume 12, pages 105974–105989 (2024).

[4] In the article titled "Secure consensus control on multi-agent systems based on improved PBFT and Raft blockchain consensus algorithms," published in IEEE/CAA J. Autom. Sinica (Volume 12, Issue 7, Pages 1407–1417) in the year 2025, with the digital object identifier 10. 1109/JAS. 2025. 125300, J. Zhu, C. Lu, J. Li, and F.-Y. Wang discuss how multi-agent systems can securely reach agreement using advanced versions of the PBFT as well as Raft consensus protocols integrated within blockchain environment

[5] In 2024, this was published in volume 12 of IEEE Access on pages 35135 to 35153: ‘Optimizing Cloud Performance: A Scheduling Approach of Microservices for Improved Fault Tolerance, Minimized Network Traffic and Reduced Latency’.

[6] The design and implementation of a CRDT-OT hybrid client-based system meant for real time collaboration editing of engineering documents is the topic of the paper authored by M. Li, C. Dang, L. Liu, and Z. Tang. This was presented at the 10th International Conference on Computer Information Processing Technologies (ISCIPT) in 2025, pages 826 to 833.

[7] L. Freudenthaler, B. Taufner, K. Göschka, (2025). On moving from characters to structure: A reexamination of real-time collaborative programming models. In 40th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE) (pp. 2695–2706). New York: IEEE.

[8] The real-time code collaboration platform in the cloud is DevSync as researched by D. Ragunath, S. Raj, I. Jackson, and P. Balaji and reported at the IEEE International Conference on Innovative Engineering and Next Generation Technologies for Sustainability (ICINVENTS) in 2025, pages 1 to 6.

[9] B M, P. Vaishnavi, S. Reddy, H. Tripathy, R. Moturi, and S. Nazma published a paper in the proceedings of the international conference on connected intelligence in industrial applications (CI2A) in 2026. The title is “An AI-powered collaborative development environment with automated workflow management”. The paper can be found on pages 1 to 7 and the digital object identifier (DOI) is 10. 1109/CI2A69097. 2026. 11576837.

[10] In the proceedings of the second International Conference on New Frontiers in Communication, Automation, Management and Security (ICCAMS) 2025, there was a paper titled “Scalable real-time chat application: Leveraging Redis and WebSocket for low-latency communication,”.

[11] In the proceedings of the second international conference on information systems and information technology (ICISIT) in 2025, a paper was presented by K. Widatama and colleagues which is titled “Development of a secure API authentication mechanism with JWT and data encryption.

[12] M. Shapiro et al. discussed conflict-free replicated data types in Stabilization, Safety, and Security of Distributed Systems (SSS 2011), Lecture Notes in Computer Science, vol. 6976 published by Springer in the year 2011.

[13] The reference was written by P. Nicolaescu, K. Jahns, M. Derntl, and R. Klamma and it appears in the proceedings of the ACM International Conference on Supporting Group Work (GROUP) for the year 2016. It discusses the topic of near real-time peer-to-peer shared editing on extensible data types.

[14] D. Ongaro and J. Ousterhout (2014) sought to find an understandable consensus algorithm as presented in the Proceedings of the USENIX Annual Technical Conference (ATC).